HEALTHCARE DATA ENGINEERING * CASE STUDY

Clinical EHR Data Cleaning & Cascading Imputation Pipeline

How I transformed 1,000 messy Electronic Health Records into an audit-compliant, zero-null clinical dataset without hallucinating patients.



Daniel .A. Dongo

Data Analyst | Python * Pandas * Healthcare Analytics

SWIPE ->

The Problem: Real-World EHR Data is Messy

Raw health records contain subtle traps that invalidate machine learning models.

1. Composite Vitals [Vitals]

* Blood pressure stored as text:
  e.g., '120/80', preventing any math.
* Whitespace and corrupted values.
* Needs clean systolic / diastolic split.

2. Dirty Data Types [Types]

* 'Age' column contained text:
  e.g., the verbal word 'forty'.
* Floating point decimals on integer years.
* Required rigorous regex & type coercion.

3. Polymorphic Dates [Dates]

* Visit dates recorded with mixed syntax:
  '2023_04_15' vs '15-Apr-2023 10:30'.
* Underscores and inconsistent formats.
* Solved with fuzzy token parsing.

4. The 'Ghost Patient' Trap [Critical]

* Blindly imputing missing records
  hallucinates artificial patients.
* Naive deletion drops 10% of real data.
* Needed a clinical triage framework.

KEY CLINICAL DILEMMA:

How do we fix 1,325 missing values across 1,000 records without fabricating synthetic patient profiles or losing statistical power?

Answer: Domain-driven triage + hierarchical cascading median imputation.

Pipeline Architecture & Workflow

A 5-stage production-grade cleaning pipeline built for clinical compliance.

Step 1: Normalization & Vital Splitting

STAGE 1

Decomposed 'blood_pressure' string into 'systolic_bp' and 'diastolic_bp'.

Normalized verbal text ('forty' -> 40) and stripped all object strings.

Step 2: Resilient Timestamp Parsing

STAGE 2

Parsed all dates with dateutil.parser (fuzzy=True) to handle underscores.

Converted 100% of records to datetime64[ns] without unhandled NaT values.

Step 3: The 'Ghost Patient' Triage Filter

STAGE 3

Audited 4 critical markers: age, blood_pressure, condition, cholesterol.

Pruned exactly 17 records (1.70%) missing >=3 markers, preserving 98.3%.

Step 4: Hierarchical Cascading Median Imputation

STAGE 4

Tier 1: (Age Bracket + Gender) Median -> Tier 2: Gender Median -> Tier 3: Global.

Preserves biological variance without distorting age-related risk curves.

Step 5: Regulatory Audit Flags & Outlier Review

STAGE 5

Created boolean provenance flags (*_imputed) mapped to 'Yes' / 'No'.

Audited physiological survivability bounds + 3.0x IQR outlier limits.

Code Snapshot: Decomposition & Triage

Production-grade vectorized operations in Python & Pandas.

```
# 1. Vectorized Blood Pressure Decomposition & Type Casting

df[['systolic_bp', 'diastolic_bp']] = df['blood_pressure'].str.split('/', expand=True)

# Strip trailing whitespace & coerce non-numeric values safely
df['systolic_bp'] = pd.to_numeric(df['systolic_bp'].str.strip(), errors='coerce')
df['diastolic_bp'] = pd.to_numeric(df['diastolic_bp'].str.strip(), errors='coerce')

# Cast to nullable integer to preserve clean schema
df[['systolic_bp', 'diastolic_bp']] = df[['systolic_bp', 'diastolic_bp']].round().astype('Int64')
```

```
# 2. Clinical Triage: Dropping Unrecoverable 'Ghost Patients'

# Define core clinical markers
critical_columns = ['age', 'blood_pressure', 'condition', 'cholesterol']

# Count missing clinical variables per patient
missing_per_row = df[critical_columns].isna().sum(axis=1)

# Prune patients missing >=3 core clinical markers (17 rows / 1.70%)
# Prevents 'inventing patients' via excessive imputation
df = df[missing_per_row < 3].copy()
```

Code Snapshot: Cascading Median Imputation

Preserving demographic and biological variation across age cohorts.

Hierarchical 3-Tier Fallback Imputation (Age Group + Gender)

```
# Core vitals and lab measurements requiring imputation
imputed_cols = ['systolic_bp', 'diastolic_bp', 'cholesterol']

for col in imputed_cols:
    # 1. Coerce to numeric
    df[col] = pd.to_numeric(df[col], errors='coerce')

    # 2. Calculate cascading fallback medians
    subgroup_med = df.groupby(['age_group', 'gender'], observed=False)[col].transform('median')
    gender_med   = df.groupby('gender', observed=False)[col].transform('median')
    global_med   = df[col].median()

    # 3. Apply cascade: Subgroup -> Gender -> Population Global
    df[col] = df[col].fillna(subgroup_med).fillna(gender_med).fillna(global_med)

# Cast to nullable Int64 and update regulatory audit flags (*_imputed) to Yes/No
df[imputed_cols] = df[imputed_cols].round().astype('Int64')
bool_cols = df.select_dtypes(include='bool').columns
df[bool_cols] = df[bool_cols].replace({True: 'Yes', False: 'No'})
```

WHY CASCADING MEDIANS MATTER:

A 25-year-old female and a 75-year-old male have vastly different healthy blood pressure and cholesterol ranges. Imputing by stratified age/sex brackets preserves clinical reality.

Project Results: Before vs. After

100% missing value resolution with verified clinical integrity.

1,000 -> 983

PATIENTS RETAINED

98.3% cohort preserved;
17 ghost profiles dropped.

1,325 -> 0

MISSING VALUES

100% missing data resolved
with zero nulls remaining.

10 -> 15

DATA ATTRIBUTES

+5 engineered columns:
vitals, age brackets, audit flags.

100% COMPLIANT

AUDIT PROVENANCE

Full transparency: all imputed
cells tracked with Yes/No flags.

FINAL DELIVERABLE: cleaned_healthcare_data.csv

* 983 clean rows ready for clinical predictive modeling and survival analysis.

* Numeric systolic & diastolic blood pressure columns enabled.

* 100% auditable lineage adhering to clinical regulatory standards.

Key Takeaways & GitHub Repository

What I learned and how to explore the full open-source code.

1. Domain Knowledge Trumps Generic Cleaning

Takeaway 1

* Healthcare data cannot be treated like generic e-commerce logs.
* Imputing everything blindly invents patients; listwise deletion destroys real patients.
* Triage filters based on core vitals strike the optimal statistical balance.

2. Explainable Imputation for Clinical Audits

Takeaway 2

* Stratified cascading medians are fast, 100% explainable, and zero-overhead.
* Explicit boolean audit flags (*_imputed) ensure compliance with HIPAA and FDA 21 CFR.
* Critical for medical AI models where training data provenance is scrutinized.

EXPLORE THE FULL PROJECT ON GITHUB:

https://github.com/ELSHINE123/healthcare-data-cleaning-pipeline

Developed by: Daniel .A. Dongo

LinkedIn: linkedin.com/in/daniel-dongo

STAR THE REPO *